

Manual de Uso e Implementación: AudioManager

El **AudioManager** es un sistema centralizado mediante el patrón Singleton que gestiona la reproducción de música, efectos de sonido (SFX) y sonidos de interfaz. Creado para persistir entre escenas gracias a **DontDestroyOnLoad**.

Canales de Audio Disponibles (AudioSources)

El sistema divide los sonidos del juego en 5 canales dedicados, cada uno con una función específica:

- **Music (**musicSource**)**: Dedicado exclusivamente a las pistas musicales de fondo. Transiciona suavemente (fade in/out) entre canciones.
- **Ambient (**ambientSource**)**: Sonidos ambientales generales y constantes que se repiten en bucle (ej: lluvia constante, viento, sonido de ciudad de fondo).
- **Ambient Spot (**ambientSpotSource**)**: Efectos ambientales localizados o específicos que actúan como capa secundaria y se repiten en bucle (ej: crepitar de una fogata en la escena o maquinaria de fondo).
- **SFX (**sfxSource**)**: Todos los efectos de sonido de un solo uso en el juego (ej: disparos, saltos, impactos de daño, pasos). No interrumpen otros audios del mismo canal.
- **UI (**uiSource**)**: Efectos sonoros exclusivos de la interfaz de usuario (ej: clics en botones, abrir/cerrar menús, notificaciones).

1. Configuración Inicial e Implementación con Prefab

El **AudioManager** ya cuenta con su propio **Prefab** preconfigurado en el proyecto. Dado que es un **Singleton persistente** (utiliza **DontDestroyOnLoad**), **solo debe instanciarse una vez** durante toda la ejecución del juego.

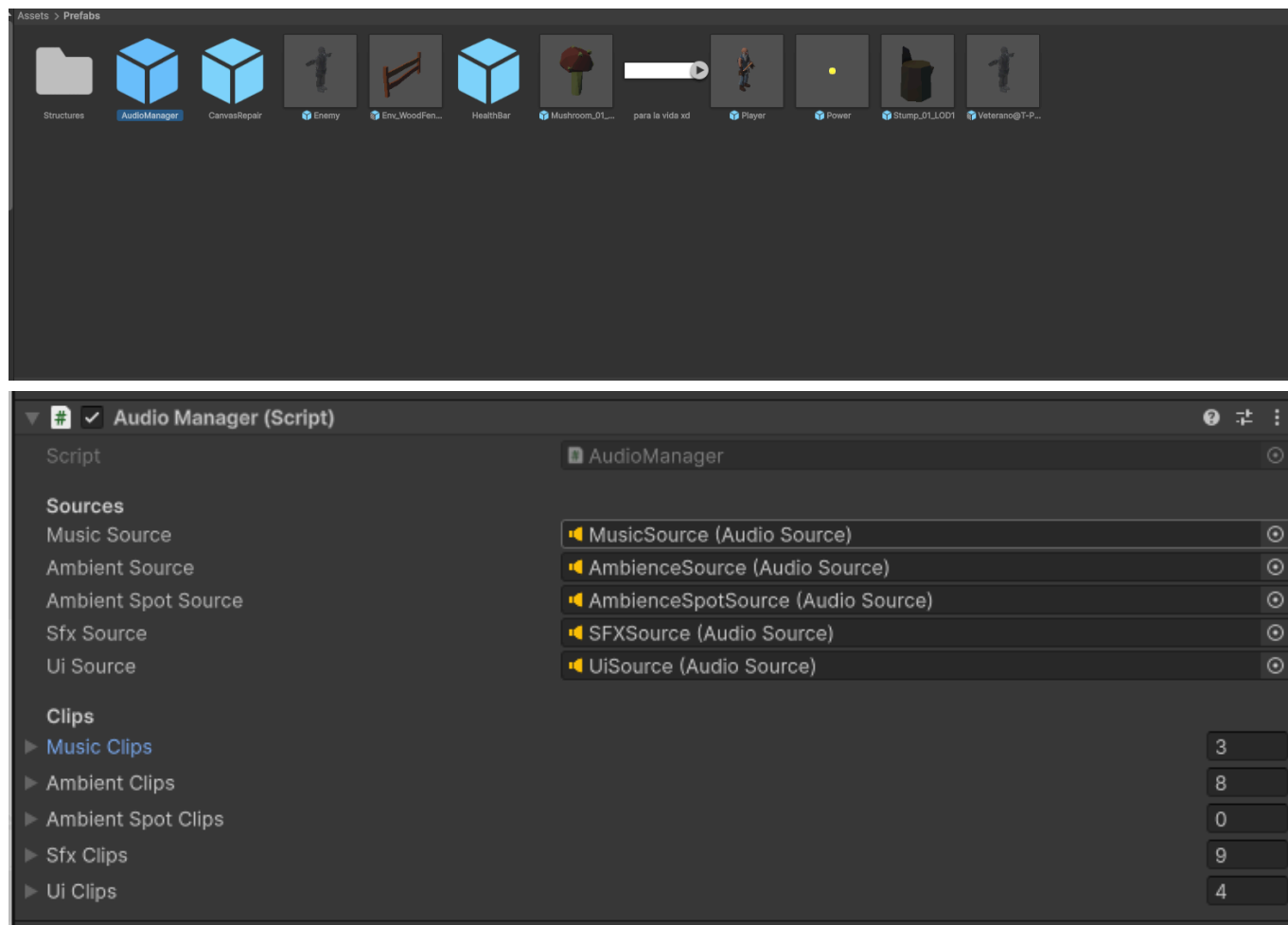
Flujo Principal (Juego Completo)

Para el flujo normal del juego, solo necesitas colocar el Prefab del **AudioManager** **únicamente en la primera escena de arranque** (por ejemplo, el Splash Screen, Carga o Menú Principal). Al hacer transiciones a otros niveles o escenas, el **AudioManager** sobrevivirá automáticamente, permitiendo que la música u otros sonidos no se corten abruptamente.

Entorno de Pruebas (Testeo de Niveles Aislados)

Si estás probando o diseñando una escena avanzada directamente (sin pasar por el menú inicial) y necesitas probar los sonidos, sigue estos pasos:

1. Arrastra el **Prefab del AudioManager** temporalmente a tu escena.
2. Si un nivel carga y ya existía un **AudioManager** previo, el script está programado para destruir el nuevo de forma automática (`if (Instance != null) { Destroy(gameObject); return; }`), evitando conflictos.
3. *Buena práctica*: Puedes dejar el prefab en todas las escenas para facilitar el testeo aislado, siempre y cuando todos usen el mismo prefab y las mismas listas actualizadas de sonidos, aunque típicamente se recomienda tener una escena de "Boot" o inicio que lo cargue primero.

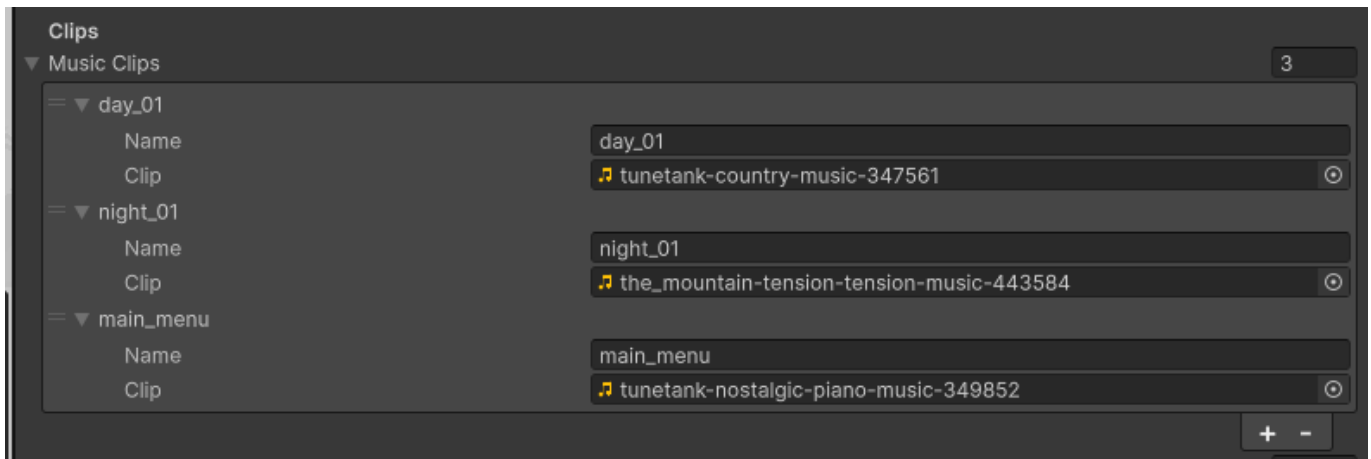


2. Asignación de Clips de Audio

El **AudioManager** maneja diccionarios internamente, pero de cara al usuario en el editor se configuran mediante listas de la estructura **SoundEntry**. Estas listas vinculan un nombre (**string**) con un recurso de audio (**AudioClip**).

Encontrarás las siguientes listas (**musicClips**, **ambientClips**, **ambientSpotClips**, **sfxClips**, **uiClips**). Para registrar un sonido:

1. Expande la lista deseada y aumenta su tamaño o presiona **+**.
2. Arrastra el recurso **AudioClip** al campo **clip**.
3. (Opcional) Asignar un **name**.
 - Si introduces un **name**, ese será el nombre que usarás en el código para llamarlo.
 - Si dejas el campo **name** en blanco, el sistema tomará automáticamente el nombre del recurso de archivo **AudioClip** como identificador.



3. Uso en el Código (API)

Al estar definido como un Singleton persistente (`AudioManager.Instance`), puedes invocar sus métodos públicos desde cualquier otro script sin necesidad de asignaciones adicionales.

Reproducir SFX y UI (Sonidos de una Sola Reproducción)

Estos métodos usan la función `PlayOneShot` para que no corten los sonidos que ya se estén reproduciendo por el mismo canal.

```
// Reproduce un sonido registrado en la lista de SFX
AudioManager.Instance.PlaySFX("nombre_sfx");

// Reproduce un sonido registrado en la lista de Interfaz de Usuario
AudioManager.Instance.PlayUI("nombre_click_boton");
```

Reproducir Música (Con Fundido)

La reproducción de música incorpora una corrutina que reduce automáticamente el volumen de la canción actual (Fade Out) e incrementa el volumen de la canción entrante (Fade In).

```
// Iniciar música usando la duración por defecto de desvanecimiento (1 segundo)
AudioManager.Instance.PlayMusic("nombre_cancion");

// Iniciar música e indicar explícitamente cuánto dura el fundido
AudioManager.Instance.PlayMusic("nombre_cancion", 2.5f);

// Detener la música (incluye el efecto de fundido)
AudioManager.Instance.StopMusic();
```

Reproducir Sonidos Ambientales

Los sonidos de ambiente se configuran automáticamente para reproducirse de forma continua (`loop = true`).

```
AudioManager.Instance.PlayAmbience("nombre_lluvia");
AudioManager.Instance.PlayAmbientSpot("nombre_antorcha");

// Detener forzosamente
AudioManager.Instance.StopAmbience();
AudioManager.Instance.StopUI(); // Detiene los canales de interfaz
```

Control de Volumen y UI

El sistema cuenta con un método de apoyo diseñado explícitamente para controlar el volumen desde un Slider en un Canvas, asegurando que el valor introducido no salga de los límites de 0 a 1.

```
// Ejemplo: Controlando el volumen del contenedor SFX desde una barra deslizable
AudioManager.Instance.ChangeVolumeWithSlider(AudioManager.Instance.sfxSource,
slider.value);
```

```
0 references
public void OnPointerEnter(PointerEventData eventData)
{
    // Implementacion del audiomanager, para el sonido de hover
    AudioManager.Instance.PlayUI("hover_sound");

    // Activa outline del sprite
    outline.enabled = true;

    // Texto negro con outline blanco
    if (label != null)
    {
        label.color = Color.black;
        label.outlineWidth = 0.2f;
    }

    // Pequeño pop
    transform.localScale = originalScale * 1.05f;
}
```